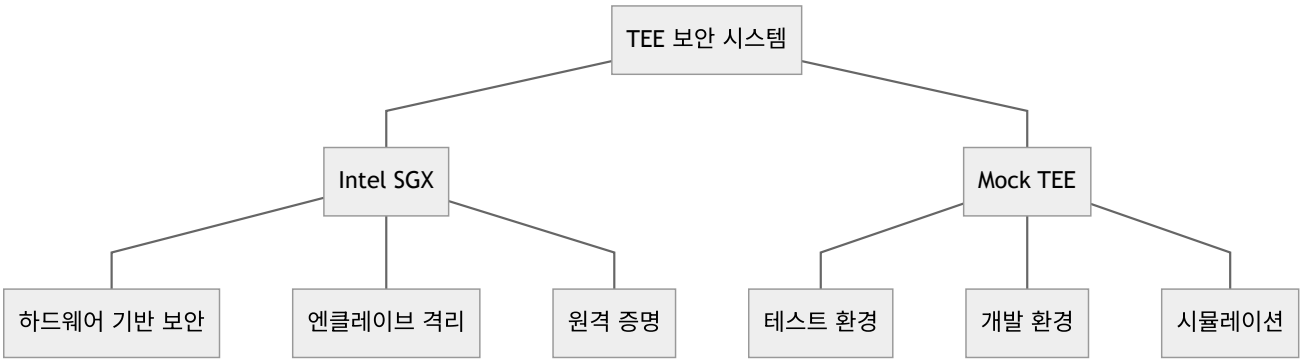
# TEE 보안 (Trusted Execution Environment Security)

## 소개

Oasis Core는 Trusted Execution Environment(TEE)를 활용하여 런타임 실행의 기밀성과 무결성을 보장합니다. 주로 Intel SGX(Software Guard Extensions)를 사용하여 엔클레이브 기반의 보안 실행 환경을 제공하며, 이를 통해 민감한 데이터와 계산을 보호합니다.

## TEE 아키텍처 개요

### 1. TEE 종류별 분류




### 2. TEE 선택 모드

```
// TEESelectMode는 TEE 선택 모드를 정의합니다

type TEESelectMode string
```

```go
const (
TEESelectModeAuto TEESelectMode = "auto"  // 자동 선택
TEESelectModeHardware TEESelectMode = "hardware"  // 하드웨어 TEE 강제
TEESelectModeNone TEESelectMode = "none"  // TEE 비활성화
)

// TEEKind는 지원되는 TEE 종류를 정의합니다
type TEEKind uint8

const (
TEEKindNone TEEKind = 0  // TEE 없음
TEEKindSGX TEEKind = 1  // Intel SGX
)
```

# Intel SGX 엔클레이브 구조

## 1. 엔클레이브 신원 관리

### MRENCLAVE와 MRSIGNER

```go
// MrEnclave는 SGX 엔클레이브 신원 레지스터 값입니다
type MrEnclave [MrEnclaveSize]byte

// MrSigner는 SGX 엔클레이브 서명자 신원 값입니다
```

```go
type MrSigner [MrSignerSize]byte


// EnclaveIdentity는 MRSIGNER/MRENCLAVE 쌍입니다
type EnclaveIdentity struct {
MrEnclave MrEnclave `json:"mr_enclave"`
MrSigner MrSigner `json:"mr_signer"`
}
```

### 엔클레이브 속성

```go
// AttributesFlags는 엔클레이브 리포트 속성 플래그입니다
type AttributesFlags uint64


const (
AttributeInit AttributesFlags = 0b0000_0001  // 초기화됨
AttributeDebug AttributesFlags = 0b0000_0010  // 디버그 모드
AttributeMode64Bit AttributesFlags = 0b0000_0100  // 64비트 모드
AttributeProvisionKey AttributesFlags = 0b0001_0000  // 프로비저닝 키
AttributeEInitTokenKey AttributesFlags = 0b0010_0000  // 초기화 토큰 키
)


// Attributes는 SGX 엔클레이브 속성 값입니다
type Attributes struct {
```

```go
Flags AttributesFlags // 속성 플래그
Xfrm  uint64  // 확장 기능 요청 마스크
}

// Contains는 주어진 플래그 속성이 포함되어 있는지 확인합니다
func (a AttributesFlags) Contains(flag AttributesFlags) bool {
return (uint64(a) & uint64(flag)) != 0
}
```

## 2. SGX 프로비저너 구조

### SGX 설정

```go
// Config는 SGX 전용 프로비저너 설정 옵션을 포함합니다
type Config struct {
// HostInfo는 호스트 환경 정보를 제공합니다
HostInfo *protocol.HostInfo
// CommonStore는 노드의 공통 영구 저장소 핸들입니다
CommonStore *persistent.CommonStore
// LoaderPath는 런타임 로더 바이너리 경로입니다
LoaderPath  string
// IAS는 Intel Attestation Service 엔드포인트입니다
IAS []ias.Endpoint
// PCS는 Intel Provisioning Certification Service입니다
```

```
PCS pcs.QuoteService

// Consensus는 합의 계층 백엔드입니다

Consensus consensus.Service

// Identity는 노드 신원입니다

Identity *identity.Identity

// RuntimeAttestInterval은 주기적 런타임 재증명 간격입니다

RuntimeAttestInterval time.Duration

// 보안 설정

InsecureNoSandbox  bool  // 샌드박스 비활성화 (테스트용)

InsecureMock  bool  // 모의 SGX 모드 (테스트용)

}
```

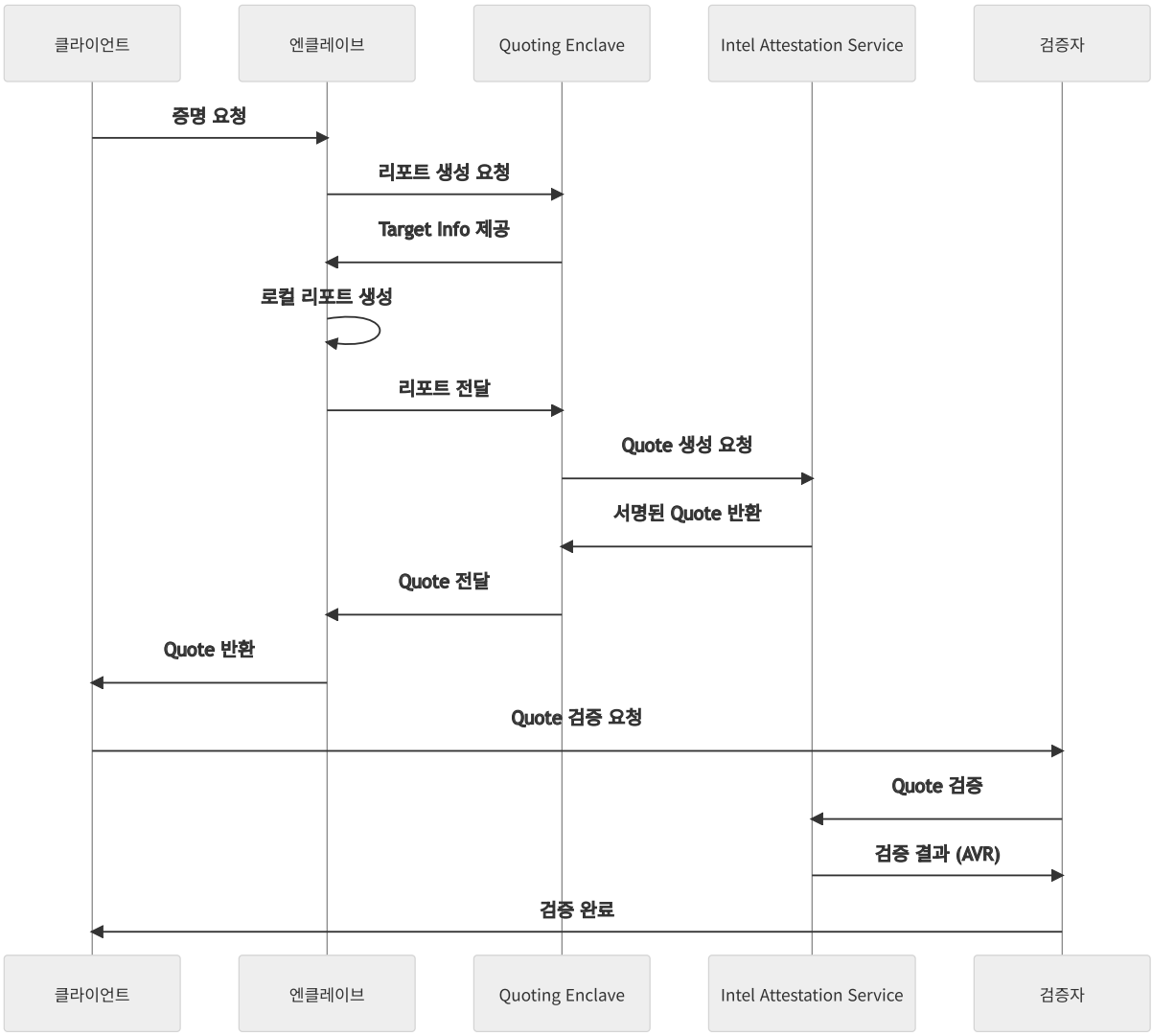
### SGX 프로비저너 구현

```
type sgxProvisioner struct {

sync.Mutex

cfg Config

sandbox host.Provisioner

ias []ias.Endpoint

pcs pcs.QuoteService

aesm *aesm.Client

consensus consensus.Service

identity *identity.Identity

store *persistent.CommonStore
```

```
logger *logging.Logger

}
```

# 원격 증명 (Remote Attestation) 메커니즘

## 1. 증명 프로세스 개요




## 2. Quote 생성과 검증

### Quote 구조

```go
// Quote는 SGX 원격 증명 Quote를 나타냅니다
type Quote struct {
// Quote 헤더 정보
Version  uint16
SignType  uint16
EPIDGroupID [4]byte
QESVNVersion  uint16
PCESVNVersion  uint16
// 엔클레이브 리포트
Report Report
// 서명 데이터
SignatureLen  uint32
Signature []byte
}


// Report는 SGX 엔클레이브 리포트입니다
type Report struct {
CPUSVNVersion [16]byte  // CPU 보안 버전
MiscSelect  uint32  // 기타 선택 비트
Attributes Attributes // 엔클레이브 속성
MrEnclave MrEnclave // 엔클레이브 측정값
MrSigner MrSigner // 서명자 측정값
```

```
ISVProdID  uint16  // ISV 제품 ID
ISVSVNVersion  uint16  // ISV 보안 버전
ReportData [64]byte  // 사용자 데이터
}
```

### Quote 검증 프로세스

```
// Verify는 SGX 원격 증명 Quote를 검증합니다
func (q *Quote) Verify(policy *QuotePolicy, ts time.Time) (*sgx.VerifiedQuote, e
// 1. Quote 구조 검증
if  err := q.validateStructure();  err != nil {
return  nil, fmt.Errorf("Quote 구조 검증 실패: %w", err)
}
// 2. 정책 기반 검증
if  err := q.verifyPolicy(policy);  err != nil {
return  nil, fmt.Errorf("정책 검증 실패: %w", err)
}
// 3. 서명 검증
if  err := q.verifySignature(ts);  err != nil {
return  nil, fmt.Errorf("서명 검증 실패: %w", err)
}
// 4. TCB 레벨 검증
tcbLevel, err := q.verifyTCBLevel(policy, ts)
if  err != nil {
```

```go
return nil, fmt.Errorf("TCB 레벨 검증 실패: %w", err)
}
return &sgx.VerifiedQuote{
Report: q.Report,
TCBLevel: tcbLevel,
}, nil
}
```

## 3. Intel Attestation Service (IAS) 연동

### AVR (Attestation Verification Report) 처리

```go
// AVRBundle은 IAS에서 반환된 증명 검증 리포트 번들입니다
type AVRBundle struct {
Body []byte `json:"body"` // AVR 본문
Signature []byte `json:"signature"` // IAS 서명
CertificateChain [][]byte `json:"certificate_chain"` // 인증서 체인
}

// Open은 AVR 번들을 디코딩하고 검증합니다
func (b *AVRBundle) Open(policy *QuotePolicy, trustRoots *x509.CertPool, ts time
// 1. 인증서 체인 검증
if err := b.verifyCertificateChain(trustRoots, ts); err != nil {
return nil, fmt.Errorf("인증서 체인 검증 실패: %w", err)
```

```go
}

// 2. IAS 서명 검증

if err := b.verifySignature(); err != nil {

return nil, fmt.Errorf("IAS 서명 검증 실패: %w", err)

}

// 3. AVR 파싱

var avr AttestationVerificationReport

if err := json.Unmarshal(b.Body, &avr); err != nil {

return nil, fmt.Errorf("AVR 파싱 실패: %w", err)

}

// 4. Quote 상태 검증

if !avr.quoteStatusAllowed(policy) {

return nil, fmt.Errorf("허용되지 않은 Quote 상태: %s", avr.ISVEnclaveQuoteStatus)

}

return &avr, nil

}
```

## 4. PCS (Provisioning Certification Service) 지원

### ECDSA 기반 증명

```go
// QuoteBundle은 PCS 기반 Quote 번들입니다

type QuoteBundle struct {
```

```go
Quote []byte `json:"quote"` // ECDSA Quote
TCBInfoIssuerChain []byte `json:"tcb_info_issuer_chain"` // TCB 정보 발급자 체인
QEIdentityIssuerChain []byte `json:"qe_identity_issuer_chain"` // QE 신원 발급자
}

// Verify는 PCS Quote 번들을 검증합니다
func (bnd *QuoteBundle) Verify(policy *QuotePolicy, ts time.Time) (*sgx.Verified
var quote Quote
if err := quote.UnmarshalBinary(bnd.Quote); err != nil {
return nil, fmt.Errorf("Quote 파싱 실패: %w", err)
}
// TCB 번들 검증
tcbBundle, err := bnd.getTCBBundle(policy, ts)
if err != nil {
return nil, fmt.Errorf("TCB 번들 검증 실패: %w", err)
}
return quote.Verify(policy, ts, tcbBundle)
}
```

## TEE 보안 API와 증명 예제

### 1. 런타임 TEE 능력 초기화

#### RAK (Runtime Attestation Key) 초기화

```go
// RuntimeCapabilityTEERakInitRequest는 TEE 능력 초기화 요청입니다
type RuntimeCapabilityTEERakInitRequest struct {
TargetInfo []byte  `json:"target_info"`
}

// RuntimeCapabilityTEERakReportResponse는 RAK 리포트 응답입니다
type RuntimeCapabilityTEERakReportResponse struct {
RakPub signature.PublicKey `json:"rak_pub"`  // RAK 공개키
RekPub *x25519.PublicKey `json:"rek_pub,omitempty"`  // REK 공개키 (선택사항)
Report []byte  `json:"report"`  // SGX 리포트
Nonce  string  `json:"nonce"`  // 논스
}
```

### TEE 증명 워커

```go
// AttestationWorker는 주기적 증명을 수행합니다
func  AttestationWorker(
ctx context.Context,
runtimeID common.Namespace,
rt host.Runtime,
interval time.Duration,
logger *logging.Logger,
) {
```

```
ticker := time.NewTicker(interval)
defer ticker.Stop()
for {
select {
case <-ctx.Done():
return
case <-ticker.C:
if err := performAttestation(ctx, runtimeID, rt, logger); err != nil {
logger.Error("증명 실패", "error", err)
UpdateAttestationMetrics(runtimeID, component.TEEKindSGX, err)
} else {
logger.Debug("증명 성공")
UpdateAttestationMetrics(runtimeID, component.TEEKindSGX, nil)
}
}
}
}


func performAttestation(ctx context.Context, runtimeID common.Namespace, rt hos
// 1. Target Info 요청
targetInfo, err := rt.GetTargetInfo(ctx)
if err != nil {
return fmt.Errorf("Target Info 요청 실패: %w", err)
```

```
}

// 2. 리포트 생성 요청

rakReq := &RuntimeCapabilityTEERakInitRequest{

TargetInfo: targetInfo,

}

response, err := rt.Call(ctx, &protocol.Body{

RuntimeCapabilityTEERakInitRequest: rakReq,

})

if err != nil {

return fmt.Errorf("RAK 초기화 실패: %w", err)

}

// 3. Quote 생성

quote, err := generateQuote(response.RuntimeCapabilityTEERakReportResponse.Repor

if err != nil {

return fmt.Errorf("Quote 생성 실패: %w", err)

}

// 4. Quote 검증

return verifyQuote(quote)

}
```

## 2. 키 매니저 엔클레이브 증명

### 엔클레이브 정책 관리

```go
// PolicySGX는 SGX 키 매니저 접근 제어 정책입니다
type PolicySGX struct {
Serial  uint32  `json:"serial"`  // 정책 일련번호
// ID는 키 매니저 런타임 ID입니다
ID common.Namespace `json:"id"`
// Enclaves는 SGX 키 매니저 엔클레이브별 정책 맵입니다
Enclaves  map[sgx.EnclaveIdentity]*EnclavePolicySGX `json:"enclaves,omitempty"`
// MayQuery는 개인키 자료를 쿼리할 수 있는 런타임 ID 맵입니다
MayQuery  map[common.Namespace][]sgx.EnclaveIdentity `json:"may_query,omitempty"
// MayReplicate는 복제를 수행할 수 있는 엔클레이브 ID 벡터입니다
MayReplicate []sgx.EnclaveIdentity `json:"may_replicate,omitempty"`
}


// EnclavePolicySGX는 SGX 키 매니저 엔클레이브별 접근 제어 정책입니다
type EnclavePolicySGX struct {
// MayQuery는 개인키 자료를 쿼리할 수 있는 엔클레이브 ID 맵입니다
MayQuery  map[common.Namespace][]sgx.EnclaveIdentity `json:"may_query,omitempty"
// MayReplicate는 복제를 수행할 수 있는 엔클레이브 ID 벡터입니다
MayReplicate []sgx.EnclaveIdentity `json:"may_replicate,omitempty"`
}
```

### 서명된 정책 검증

```go
// SignedPolicySGX는 서명된 SGX 키 매니저 접근 제어 정책입니다
type SignedPolicySGX struct {
Policy PolicySGX `json:"policy"`
Signatures []signature.RawSignature `json:"signatures"`
}


// Verify는 서명된 정책을 검증합니다
func (sp *SignedPolicySGX) Verify(trustRoot signature.PublicKey) error {
// 1. 정책 직렬화
policyBytes := cbor.Marshal(sp.Policy)
// 2. 각 서명 검증
for i, sig := range sp.Signatures {
if err := signature.Verify(trustRoot, PolicySignatureContext, policyBytes, sig[
return fmt.Errorf("서명 %d 검증 실패: %w", i, err)
}
}
return nil
}
```

## 3. 실제 사용 예제

### 노드 설정에서 TEE 활성화

```yaml
# oasis-node 설정 파일
runtime:
# 런타임 설정
paths:
- /opt/oasis/runtimes/simple-keyvalue.orc
# SGX 설정
sgx:
# SGX 로더 경로
loader: /opt/oasis/bin/oasis-core-runtime-loader
# 증명 설정
attestation:
# 증명 간격 (기본: 2시간)
interval: "2h"
# IAS 설정
ias:
- api_key: "${IAS_API_KEY}"
spid: "${IAS_SPID}"
endpoint: "https://api.trustedservices.intel.com/sgx/dev/attestation/v4"
# PCS 설정 (DCAP)
pcs:
endpoint: "https://api.trustedservices.intel.com/sgx/certification/v4"
# 보안 설정
```

```yaml
debug: false  # 프로덕션에서는 false
require_attestation: true  # 증명 필수
```

### SGX 엔클레이브 빌드 스크립트

```bash
#!/bin/bash
# SGX 엔클레이브 빌드 스크립트

set -euo pipefail

ENCLAVE_DIR="/opt/oasis/enclave"
RUNTIME_DIR="/opt/oasis/runtime"

# SGX SDK 환경 설정
source /opt/intel/sgxsdk/environment

echo "엔클레이브 빌드 시작..."

# 1. 엔클레이브 소스 컴파일
cd "${ENCLAVE_DIR}"
make clean
make SGX_MODE=HW SGX_PRERELEASE=0

# 2. 서명 키 생성 (개발용)
```

```
if [ ! -f "enclave_private.pem" ]; then
openssl genrsa -out enclave_private.pem -3 3072
fi

# 3. 엔클레이브 서명
sgx_sign sign \
-key enclave_private.pem \
-enclave enclave.so \
-out enclave.signed.so \
-config enclave.config.xml

# 4. MRENCLAVE 계산
MRENCLAVE=$(sgx_sign dump -enclave enclave.signed.so -dumpfile /dev/stdout
echo "MRENCLAVE: ${MRENCLAVE}"

# 5. 런타임 번들 생성
mkdir -p "${RUNTIME_DIR}"
cp enclave.signed.so "${RUNTIME_DIR}/"
cp manifest.json "${RUNTIME_DIR}/"

echo "엔클레이브 빌드 완료!"
```

### Quote 검증 클라이언트 예제

```go
package main


import (
"context"
"fmt"
"time"
"github.com/oasisprotocol/oasis-core/go/common/sgx/quote"
"github.com/oasisprotocol/oasis-core/go/common/sgx/ias"
)


func main() {
// Quote 데이터 (실제로는 엔클레이브에서 받음)
quoteData := []byte{/* Quote 바이트 */}
// Quote 파싱
var q quote.Quote
if err := q.UnmarshalBinary(quoteData); err != nil {
panic(fmt.Errorf("Quote 파싱 실패: %w", err))
}
// 검증 정책 설정
policy := &quote.Policy{
// 허용되는 Quote 상태
AllowedQuoteStatuses: []ias.ISVEnclaveQuoteStatus{
```

```
ias.QuoteOK,
ias.QuoteSwHardeningNeeded,
},
// 필수 엔클레이브 속성
RequiredAttributes: sgx.Attributes{
Flags: sgx.AttributeMode64Bit,
},
// 금지된 엔클레이브 속성
ForbiddenAttributes: sgx.Attributes{
Flags: sgx.AttributeDebug,
},
}
// Quote 검증
verifiedQuote, err := q.Verify(policy, time.Now())
if  err != nil {
panic(fmt.Errorf("Quote 검증 실패: %w", err))
}
fmt.Printf("Quote 검증 성공!\n")
fmt.Printf("MRENCLAVE: %x\n", verifiedQuote.Report.MrEnclave)
fmt.Printf("MRSIGNER: %x\n", verifiedQuote.Report.MrSigner)
fmt.Printf("TCB 레벨: %s\n", verifiedQuote.TCBLevel)
}
```

# 보안 고려사항

## 1. 엔클레이브 보안

- **디버그 모드 비활성화**: 프로덕션에서는 디버그 속성 금지
- **측정값 검증**: MRENCLAVE와 MRSIGNER의 엄격한 검증
- **TCB 레벨 관리**: 최신 보안 패치 레벨 유지

## 2. 증명 보안

- **재생 공격 방지**: 논스와 타임스탬프 사용
- **인증서 체인 검증**: IAS/PCS 인증서의 철저한 검증
- **정책 업데이트**: 보안 정책의 정기적 업데이트

## 3. 키 관리 보안

- **RAK 보호**: Runtime Attestation Key의 안전한 관리
- **REK 사용**: Runtime Encryption Key를 통한 데이터 암호화
- **키 회전**: 정기적인 증명 키 회전

## 4. 운영 보안

- **모니터링**: 증명 실패 및 보안 이벤트 모니터링
- **업데이트**: SGX 드라이버와 플랫폼 소프트웨어 업데이트
- **백업**: 엔클레이브 서명 키의 안전한 백업

이러한 TEE 보안 시스템을 통해 Oasis Core는 민감한 데이터와 계산을 하드웨어 수준에서 보호하며, 원격 증명을 통해 실행 환경의 무결성을 검증할 수 있습니다.